

Sprawozdanie 5:

Projektowanie danych

Autorzy:

Michał Landmann, Paweł Wyrzykowski

1. Cel i założenia projektowe

Celem tego etapu jest przekształcenie implementacyjnego diagramu klas w projekt relacyjnej bazy danych. Zgodnie z wytycznymi, diagram klas, choć docelowo implementowany w środowisku obiektowym, może zostać zaimplementowany w relacyjnej bazie danych z zachowaniem określonych reguł transformacji.

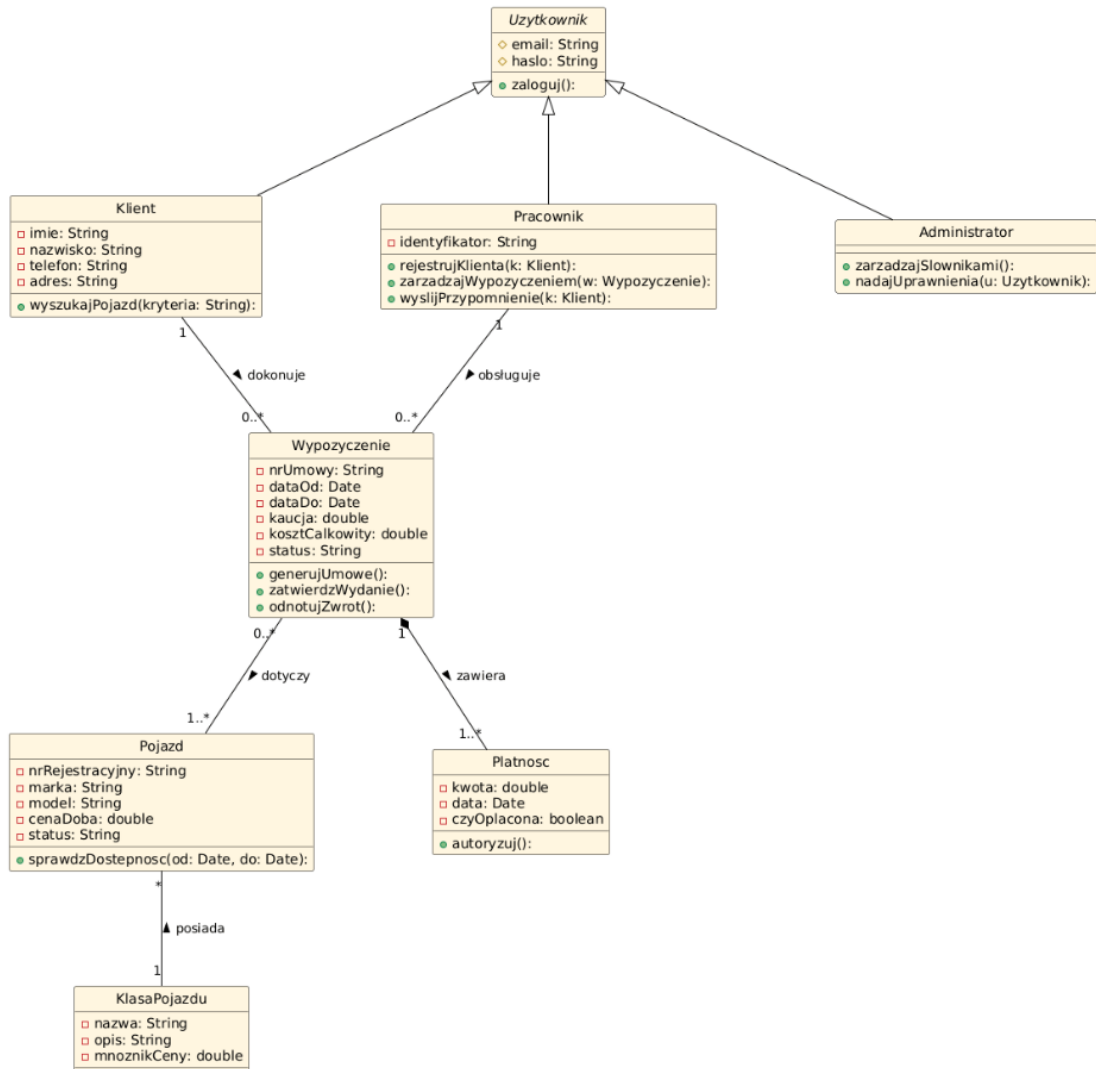
2. Implementacyjny diagram klas

Wykaz zidentyfikowanych klas to: Użytkownik, Klient (dziedziczący po Użytkowniku), Pojazd, Rezerwacja, Faktura oraz Płatność.

Zgodnie z zasadami mapowania:

- Każda klasa UML staje się tabelą w bazie danych.
- Atrybuty klasy stają się kolumnami (polami), a ich typ warunkuje typ danych w bazie.
- Operacje (metody) zostają całkowicie pominięte w schemacie bazy danych, ponieważ realizowane są po stronie logiki aplikacji.

Diagram implementacyjny Klas (SOW-MHS)



3. Projekt relacyjnej bazy danych

Zgodnie z zasadami tworzenia RBD, każda klasa staje się tabelą, a operacje (metody) zostają całkowicie pominięte, ponieważ logikę tę realizuje aplikacja.

Legenda:

- **Pogrubienie** – klucz główny (PK - Primary Key)

- *Kursywa* – klucz obcy (FK - Foreign Key)

1. Realizacja Generalizacji (Dziedziczenia)

Bazy relacyjne nie obsługują wprost dziedziczenia. Klasa bazowa i klasy pochodne stają się oddzielnymi tabelami połączonymi relacją 1:1 .

- **Uzytkownicy**(*id_uzytkownika*, email, haslo)
- **Klienci**(*id_klienta*, *id_uzytkownika*, imie, nazwisko, telefon, adres)
- **Pracownicy**(*id_pracownika*, *id_uzytkownika*, identyfikator)
- **Administratorzy**(*id_administratora*, *id_uzytkownika*)

2. Tabele Słownikowe i Katalogowe

Klasa KlasaPojazdu to słownik kategoryzujący pojazdy. Pojazd pobiera z niej dane poprzez klucz obcy (Związek 1:N).

- **Pojazdy**(*nr_rejestracyjny*, marka, klasa, model, cenaDobowa, status)
 - *Uwaga*: Jako klucz główny (PK) pojazdu użyto *nr_rejestracyjny*, gdyż jednoznacznie i naturalnie identyfikuje on obiekt.

3. Realizacja Procesu Wypożyczenia

Tabela Wypozyczenia stanowi centralny punkt systemu i łączy Pracownika z Klientem za pomocą kluczy obcych (Relacje 1:N).

- **Wypozyczenia**(*nr_umowy*, *id_klienta*, *id_pracownika*, dataOd, dataDo, kaucja, kosztCalkowity, status)

Realizacja asocjacji N:M (Wiele do Wielu): Aby zrealizować to w relacyjnej bazie danych, zgodnie z zasadami transformacji, należy wprowadzić **dodatkową tabelę pośredniczącą**.

- **Pozycje_Wypożyczenia**(*nr_umowy*, *nr_rejestracyjny*)
 - Oba pola stanowią klucz główny złożony (PK) i jednocześnie klucze obce (FK) odwołujące się do odpowiednich tabel.

4. Realizacja Kompozycji

Płatność zależy od istnienia Wypożyczenia. Zgodnie z zasadami kompozycji , usuwając lub anulując wpis w tabeli bazowej, powiązane płatności mogą zostać zablokowane lub usunięte za pomocą reguły CASCADE.

- **Płatnosci**(*id_płatnosci*, *nr_umowy*, kwota, data, czyOpłacona)
 - *nr_umowy* to klucz obcy (FK) z regułą integralności ON DELETE CASCADE.

Zgodność z normalizacją (3PN)

Zaproponowany model znajduje się w 3. Postaci Normalnej (3PN):

1. **1PN:** Wszystkie wartości w kolumnach są elementarne (atomowe), a każda tabela ma zdefiniowany klucz główny.
2. **2PN:** Każdy atrybut niekluczowy zależy od całego klucza głównego. (Rozwiązaliśmy m.in. problem przypisywania wielu aut do jednej umowy poprzez stworzenie osobnej tabeli `Pozycje_Wypozyczenia`).
3. **3PN:** Żaden atrybut niekluczowy nie zależy od innego atrybutu niekluczowego. Dane o klientach i pracownikach nie dublują się w tabeli `Wypozyczenia` – wstawiono tam wyłącznie klucze obce.